FECAP — Fundação Escola de Comércio Álvares Penteado
Análise e Desenvolvimento de Sistemas (ADS)





GEOMETRY HEROES



Pule no ritmo. Salve o dia.

Jogo de plataforma rítmica com heróis · projeto de Programação Orientada a Objetos

Integrantes
Paulo Marcelli · Pedro Henrique · Cauã Santos Nascimento · Leonardo Zachello

Conceito: um herói, uma fase, um poder

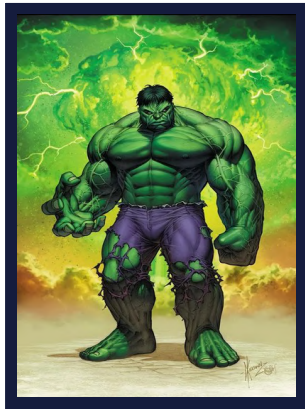
O cubo corre sozinho no ritmo da música. O jogador decide só duas coisas: quando pular e quando usar o poder do herói.



Homem-Aranha

Teia: segure no ar para balançar

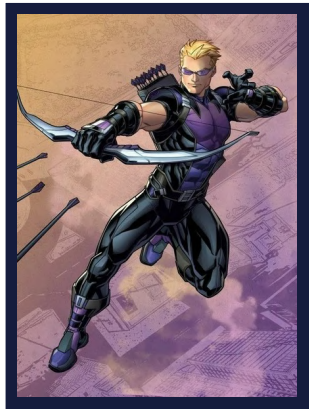
Fácil



Hulk

Fúria: cresce e atravessa o que quebra

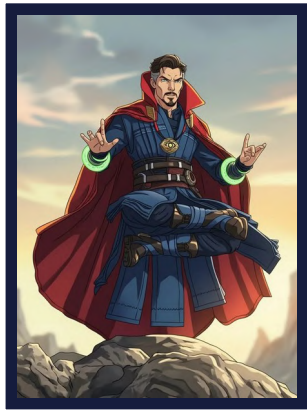
Normal



Gavião Arqueiro

Mira: acerta alvos e abre caminho

Normal



Doutor Estranho

Portais, dimensão espelho e Joia do Tempo

Difícil



Homem de Ferro

Voo: encostar no túnel mata

Insano



Feiticeira Escarlate

Realidade: troca vermelha ◆ e azul ●

Insano

Demonstração



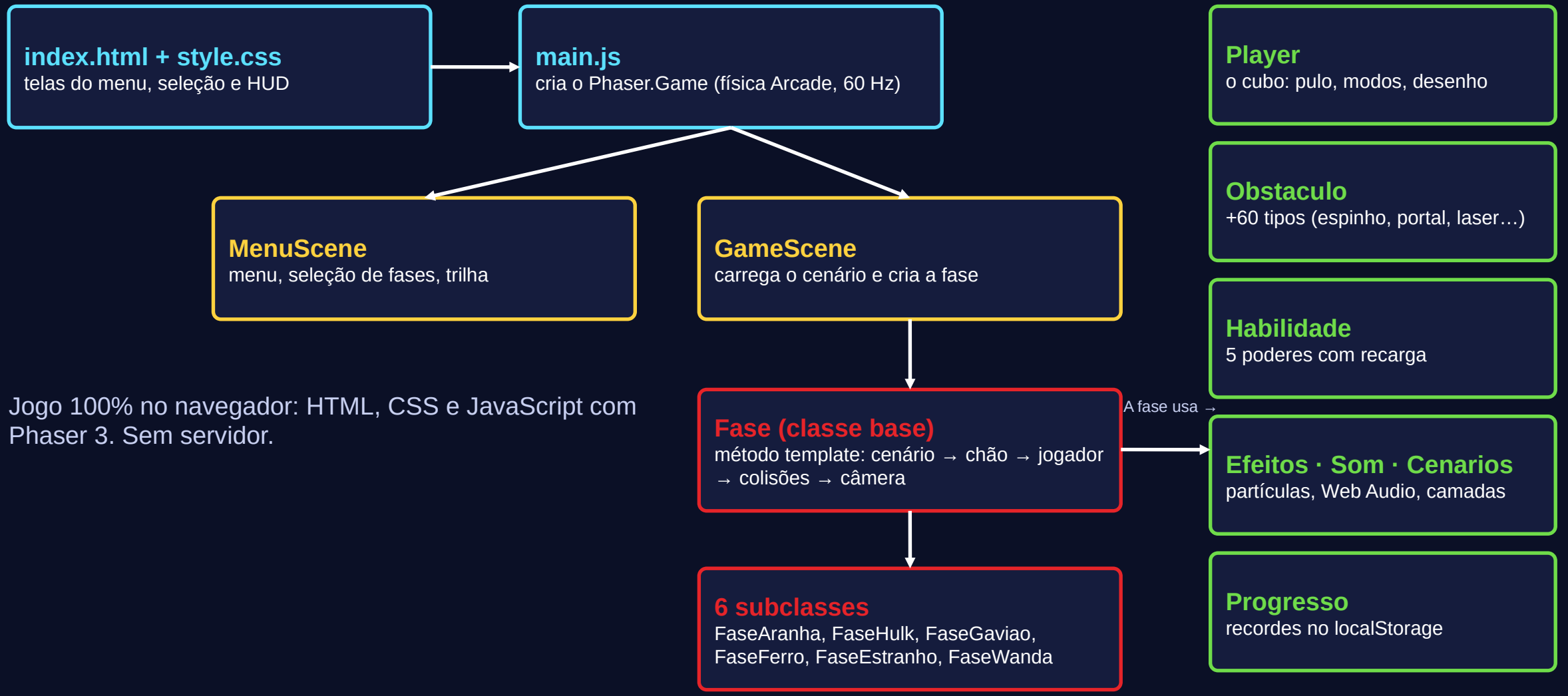
Captura real do jogo — Homem de Ferro

Como jogar

Espaço · ↑ · toque	pular (segurar = pulos seguidos)
E · Shift · botão	poder do herói
R	reiniciar
ESC	pausa
Z / X	modo prática (marcar/apagar)

Vídeo de apresentação (~2 min, 1080p, com o tema dos Vingadores) acompanha esta apresentação.

Arquitetura



Jogo 100% no navegador: HTML, CSS e JavaScript com Phaser 3. Sem servidor.

Tecnologias

Phaser 3 (v3.90)

Motor do jogo: renderização, física Arcade, câmera, partículas e tweens.

JavaScript (classes ES)

Toda a lógica em POO: 78 classes, cerca de 10.100 linhas.

HTML · CSS

Menu e seleção no estilo HQ, HUD, telas responsivas (unidades cqw).

Web Audio API

Música e efeitos sonoros sintetizados no código, sem arquivos de áudio.

Python · Playwright

Testes automáticos no Chromium sem tela, com piloto automático.

Pillow · OpenCV · ffmpeg

Preparar os cenários em camadas (WebP) e montar o vídeo.

Claude Code: como usamos

1. Pedido do grupo
2. Ler o código envolvido
3. Medir o problema com script
4. Corrigir na causa
5. Rodar os testes
6. Commit e push no GitHub

Exemplo real
O cubo "tremia" em telas de 120 Hz. Um script mediu a posição quadro a quadro (0, 7, 0, 8 px). A causa era a física em passo fixo; a solução foi desenhar e mover a câmera entre dois passos (interpolação). O tremor caiu de 8 px para 0,3 px.

78	280	7
classes	verificações	suítes de teste

O grupo definiu as regras do jogo, testou jogando e aprovou cada mudança.

Agentes de IA em paralelo



Quando usamos

Nas rodadas 7 e 8. Cada agente trabalhou numa cópia isolada do projeto (worktree) e em arquivos diferentes, para não haver conflito.

Por que não sempre

Depois, as mudanças mexiam nos mesmos arquivos (fase.js, player.js). Aí um agente só, em sequência, é mais seguro.

Manus AI e ChatGPT no fluxo

ChatGPT
ajudou o grupo a escrever os prompts

Prompt
pedido detalhado: o quê, onde e como testar

Manus AI
criou o design dos fundos de cada fase

Imagens
enviadas ao Claude pelo grupo

Claude Code
escreve o código, recorta as imagens em camadas e integra no jogo



Cada ferramenta numa etapa: o grupo escolhia o que entrava no jogo.

Design e level design



Seleção de fases no estilo HQ

- **Uma paleta por herói**
  o jogador reconhece a fase pela cor
- **Triângulo = perigo**
  nenhum triângulo decorativo
- **Fase no ritmo**
  obstáculos alinhados à batida da música
- **Aprender → combinar → dominar**
  um desafio novo por vez
- **Cenários em camadas (parallax)**
  design dos fundos criado no Manus e recortado em camadas
- **Checkpoint justo**
  pelo menos 0,8 s até o primeiro perigo

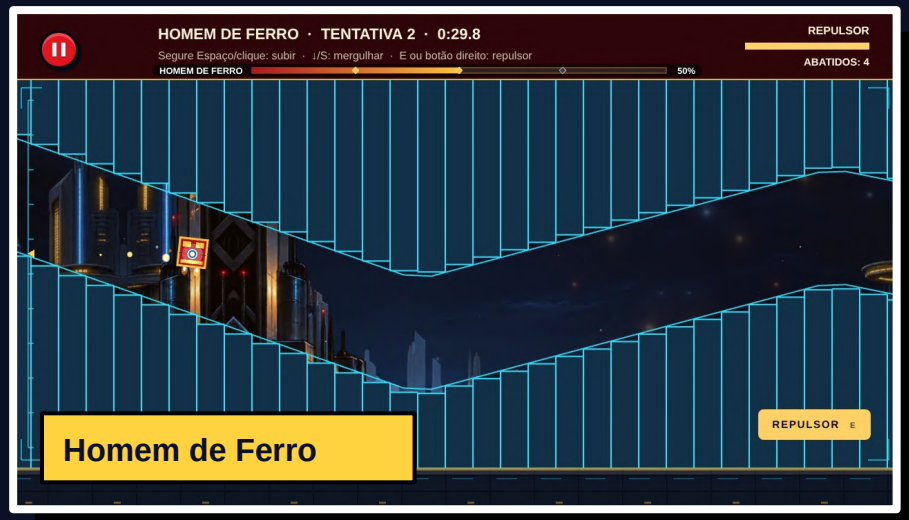
Skills de design usadas

animate

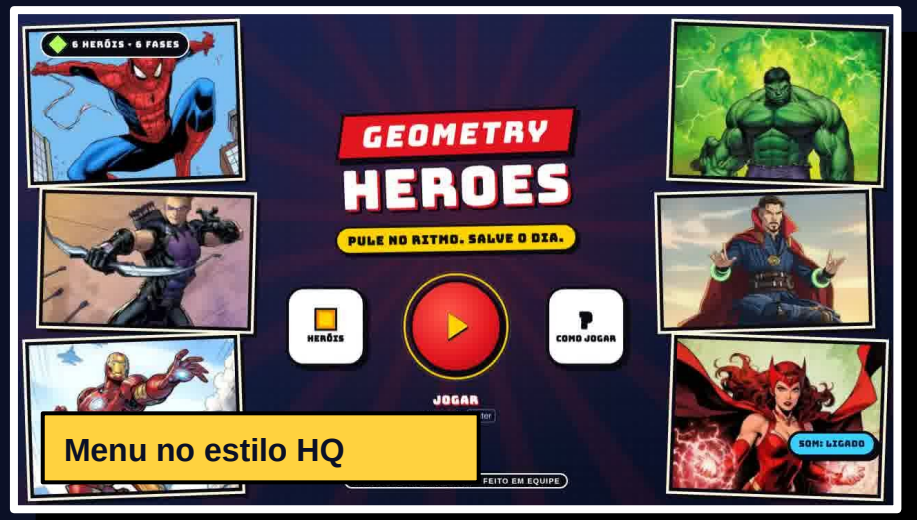
Skill de animação. Usada para decidir o que deve se mover e como: o panorama do Homem de Ferro rolando devagar e em loop, curvas ease-out e durações curtas.

emil-design-eng

Skill de engenharia de design. Usada na revisão geral de interface e movimento: legibilidade, feedback visual e detalhes que o jogador sente sem perceber.



Homem de Ferro



Menu no estilo HQ

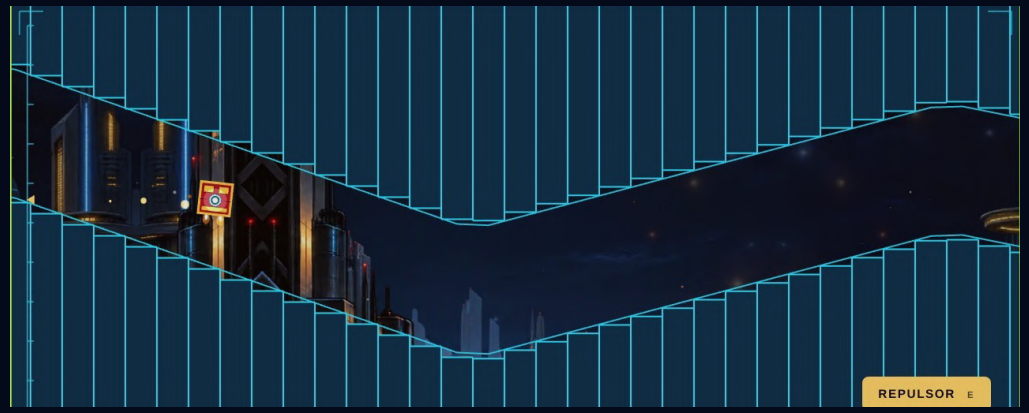
Só listamos as skills cujo uso está registrado no histórico do projeto.

Classes, objetos e atributos

A **classe** é a planta (blueprint) que define o comportamento do herói. No código real, `class Player` descreve como o cubo deve se comportar.

O **objeto** é a instância viva dessa classe. Quando a fase inicia, o código cria um novo objeto com `new`.

Os **atributos** guardam o estado atual do objeto, como se ele está **vivo**, sua **velocidadeX** ou se a gravidade está **invertida**.



Instância ativa de Player no Homem de Ferro

```
class Player extends Phaser.GameObjects.Rectangle {
  constructor(scene, x, y, cor, borda) {
    super(scene, x, y, 40, 40, cor);

    // Atributos: O Estado do Objeto
    this.velocidadeX = 450;
    this.vivo = true;
    this.invertido = false;
    this.modo = 'cubo';
  }

  decidirPulo(time, segurando) {
    // Comportamento dinâmico
    const noChao = this.isOnGround();
    // ...lógica do pulo baseada no estado
  }
}
```

Classe abstrata: a base do sistema de fases

A classe `Fase` atua como uma **classe abstrata**: ela define a estrutura comum e métodos obrigatórios, mas não deve ser jogada sozinha.

Ela fornece "ganchos" (hooks) como `checkpoints()` e `roteiro()` que devolvem valores vazios por padrão.

As fases reais (Hulk, Homem de Ferro) preenchem esses ganchos com dados específicos, aproveitando toda a mecânica de colisão e câmera da base.

NOTA TÉCNICA: Em JavaScript, não existe a palavra-chave "abstract". A abstração é conceitual e implementada via ganchos que retornam `null` ou `[]`.

```
class Fase {
  // MÉTODOS GANCHOS (devem ser sobrescritos)

  criarHabilidade() { return null; }

  roteiro() { return []; }

  checkpoints() { return []; }

  // MÉTODO TEMPLATE (lógica fixa)
  criar() {
    this.criarCenario();
    this.criarJogador();
    // Chama o gancho polimórfico:
    this.habilidade = this.criarHabilidade();
  }
}
```

Encapsulamento: estado e regras de uso

O **encapsulamento** protege a lógica interna. A classe `Habilidade` esconde a complexidade do tempo de recarga.

O mundo externo não altera o relógio diretamente; ele apenas chama `ativar()`.

A classe decide se pode ou não usar o poder, garantindo que o estado de **recarga** seja respeitado em um único lugar.

```
class Habilidade {
  constructor(fase, config) {
    this.recarga = config.recarga;
    this.ultimoUso = -Infinity;
  }

  // Proteção da Regra de Negócio
  pronta(agora) {
    return agora - this.ultimoUso ⩾ this.recarga;
  }

  ativar(agora, alvo) {
    // Validação centralizada
    if (!this.pronta(agora)) {
      return false; // BLOQUEADO
    }

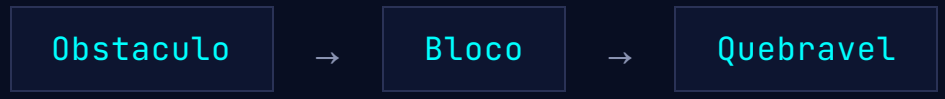
    this.usar(agora, alvo);
    this.ultimoUso = agora;
    return true;
  }
}
```

DADOS PROTEGIDOS

VALIDAÇÃO INTERNA

Herança: reaproveitar e especializar

Permite criar novas classes baseadas em classes existentes, herdando seus atributos e métodos.




```
// Superclasse: lógica de física básica
class Obstaculo {
  constructor(fase, forma, categoria) {
    this.fase = fase;
    this.ativo = true;
  }
  destruir() { ... }
}

class Bloco extends Obstaculo {
  // Reaproveita destruir() da base
}
```

```
// Subclasse: lógica específica
class Quebravel extends Bloco {
  constructor(fase, x, y, hp, tipo) {
    super(fase, x, y, ...);
    this.hp = hp;
  }

  // Método exclusivo de Quebravel
  aoAcertar(dano) {
    this.hp -= dano;
    if (this.hp ⩽ 0) this.destruir();
  }
}
```

Polimorfismo: a mesma chamada, efeitos diferentes

```
Fase chama: this.habilidade.ativar(agora);
```

```
// Na Fase do Hulk
class Furia extends Habilidade {
  usar(agora) {
    // Efeito: Crescer e atravessar
    this.fase.ligarFuria();
    return true;
  }
}
```

```
// Na Fase do H. de Ferro
class Repulsor extends Habilidade {
  usar(agora, alvo) {
    // Efeito: Disparar feixe laser
    this.disparar(alvo);
    return true;
  }
}
```

O polimorfismo permite que a classe **Fase** trate qualquer poder da mesma forma, mas cada herói reage com seu próprio efeito único ao método `usar()`.

Desafios e como resolvemos

Cubo tremendo em 120 Hz

Desenho e câmera interpolados entre passos da física: 8 px → 0,3 px.

Renascer colado no perigo

Checkpoints recuados, câmera alinhada e 0,5 s de respiro: de 0,3 s para ≥ 0,8 s.

Fúria do Hulk demorava

Espera depois da fúria caiu de 3 s para 1,5 s (ciclo de 6 s para 4,5 s).

Toque perdido no celular

Multitoque (até 3 dedos) e foco preso em botão não bloqueia mais o pulo.

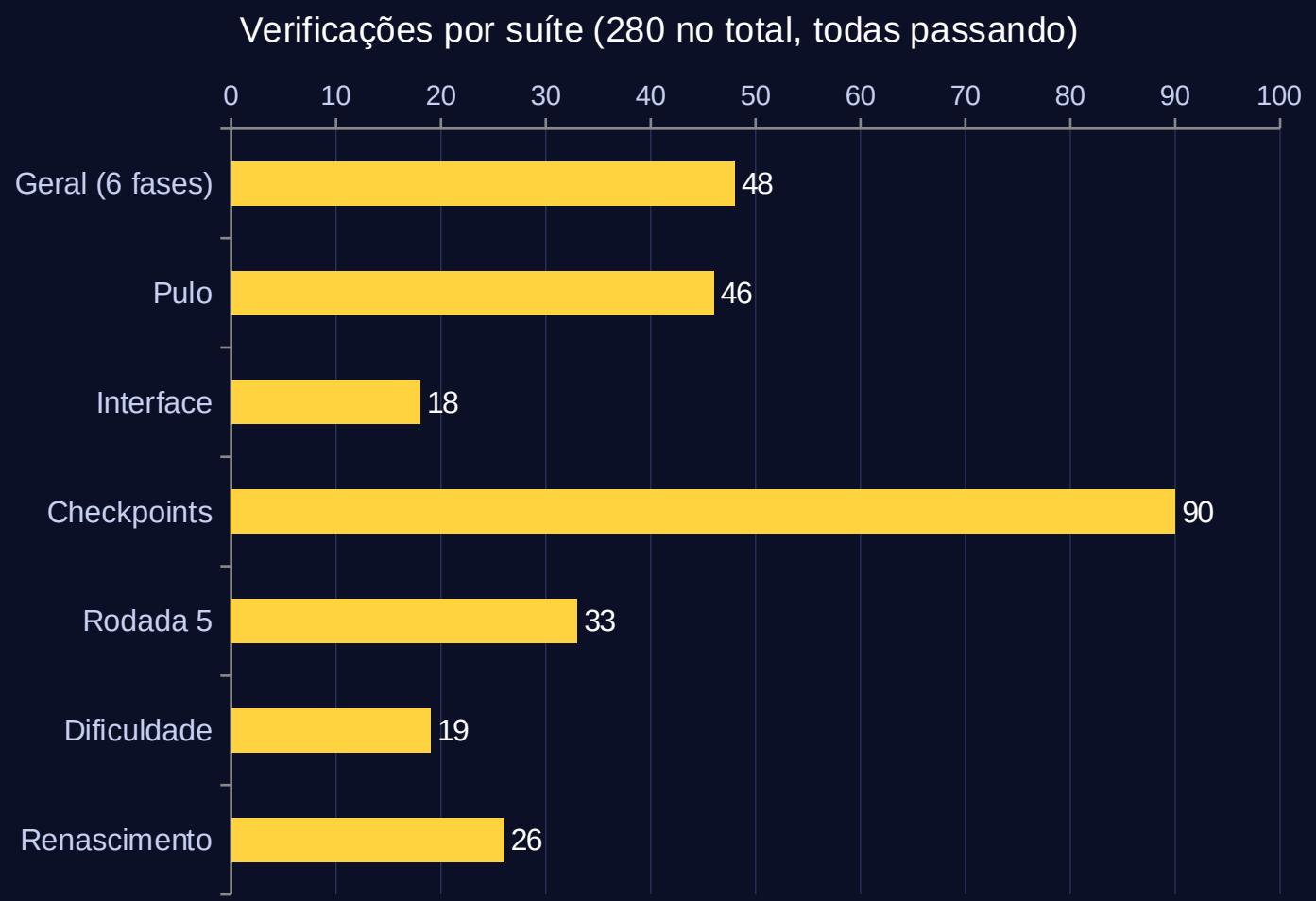
Cenário "flutuando"

Objetos desenhados por cima da arte, em outra velocidade, foram removidos.

Card da Feiticeira maior

Nome longo quebrava em 2 linhas; todos os cards ganharam a mesma altura.

Testes e correções



Como os testes jogam

O Playwright abre o jogo num Chromium sem tela e avança quadro a quadro. Um piloto automático usa os mesmos comandos do jogador (pular, poder) e precisa terminar cada fase.

Novo nesta rodada

Teste de renascimento: em cada checkpoint, morre 3 vezes seguidas e confere posição, câmera e tempo para reagir.

Resultados

6

fases completas, uma por herói

280/280

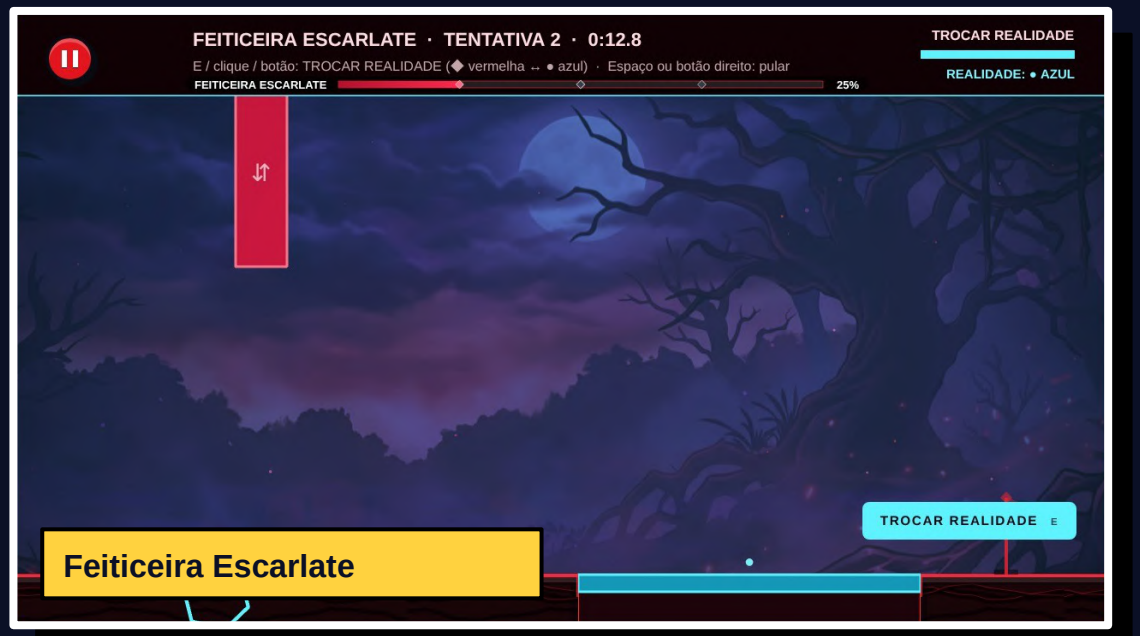
verificações passando

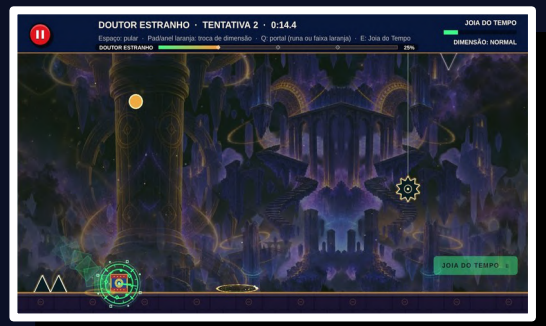
78

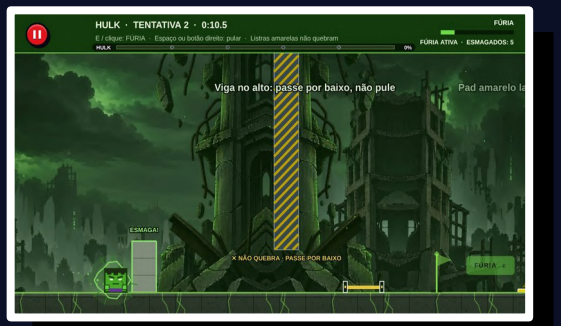
classes em JavaScript

0

arquivos de áudio: tudo sintetizado







- Jogável no navegador, no computador e no celular
- Progresso salvo no próprio navegador
- Vídeo de apresentação de ~2 min em 1080p
- Código comentado e versionado no GitHub



OBRIGADO!

Perguntas?

Paulo Marcelli · Pedro Henrique · Cauã Santos Nascimento · Leonardo Zachello

FECAP · Análise e Desenvolvimento de Sistemas